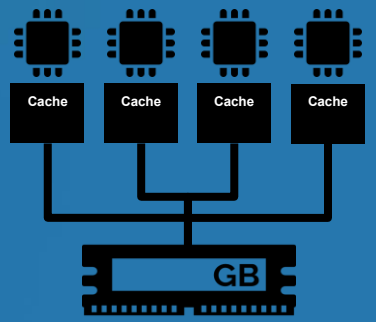
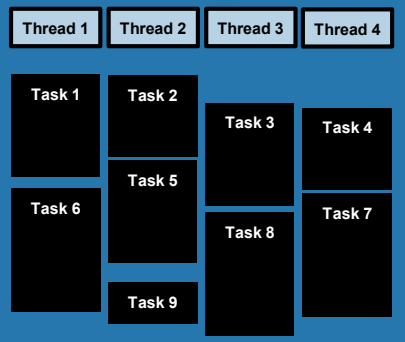
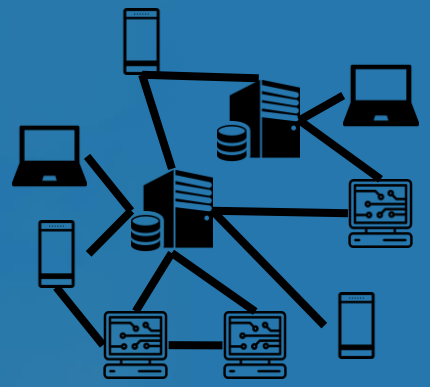
Parameterized Verification 2024







Lecture 1

Parameterized Verification 2024: Organization

- Lectures by Swen Jacobs
- Tutorials by Tom Baumeister, Shyam lal Karra, Paul Eichler

- Advanced Course, 6 CP
- Fridays 10-12
- Exercise sheets (more or less) every week

- Exams: oral exam, August 01/02 (tentative)
- Re-exams in September (negotiable)

Labeled Transition Systems

Let AP be a set of atomic propositions, i.e., observable properties of the system.

A **labeled transition system** (LTS) over AP is a tuple $(Q, Q_0, \Sigma, \delta, \lambda)$, where

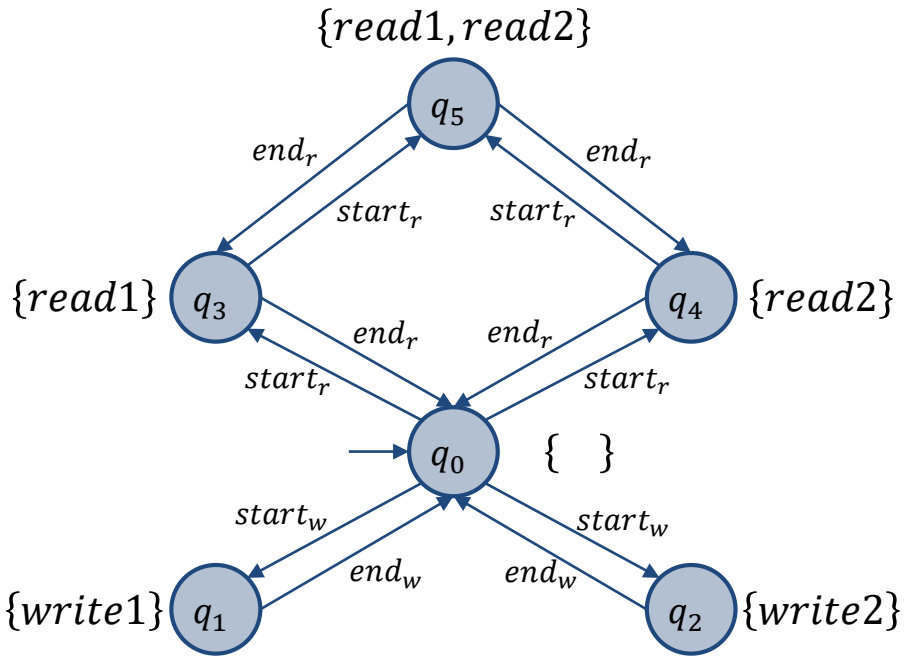
- Q is a set of states
- $Q_0 \subseteq Q$ is the set of initial states
- Σ is the set of transition labels (or actions)
- $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation
- $\lambda: Q \to 2^{AP}$ is the state labeling.

Convention: Unless explicitly stated otherwise, we consider LTSs where AP and Q are finite. Such a system is called a finite LTS.

Notation: We also write $q \xrightarrow{a} q'$ for $(q, a, q') \in \delta$.

Example: Labeled Transition System

$AP = \{read1, read2, write1, write2\}$



To improve readability, we sometimes omit transition labels or state labels if they are not important.

Run of an LTS

Consider an LTS $M = (Q, Q_0, \Sigma, \delta, \lambda)$.

A path of M an is a finite sequence $q_0 a_0 q_1 a_1 \dots q_n \in (Q\Sigma)^* Q$ or an infinite sequence $q_0 a_0 q_1 a_1 \dots \in (Q\Sigma)^\omega$ such that $q_i \xrightarrow{a_i} q_{i+1}$ for all $i < n$ or all $i \in \mathbb{N}$, respectively.

A run of M is a path of M with $q_0 \in Q_0$, and that is maximal, i.e., it cannot be extended.

A run is deadlocked if it is finite.

Depending on our specification formalism, we also consider

- state-labeled paths $q_0 q_1 \dots$, i.e., the projection of a path onto states Q,
- action-labeled paths $a_0 a_1 \dots$, i.e., the projection of a path onto actions Σ,
- traces $\lambda(q_0)\, \lambda(q_1) \dots$, i.e., the sequence of labels of a state-labeled path.

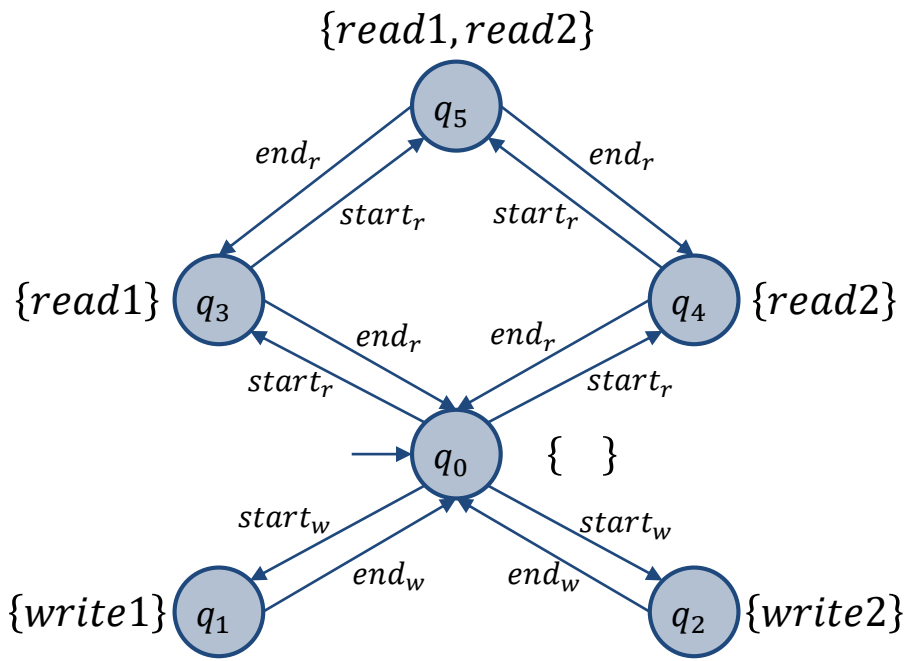
Example: Run of a System

Run: $q_0\ start_r\ q_4\ start_r\ q_5\ end_r\ q_3\ end_r q_0\ start_w\ q_1 \ldots$

State-labeled run: $q_0\ q_4\ q_5\ q_3\ q_0\ q_1 \ldots$

Action-labeled run: $start_r\ start_r\ end_r\ end_r\ start_w \ldots$

Trace: $\{\quad\}\ \{read2\}\ \{read1, read2\}\ \{read1\}\ \{\quad\}\ \{write1\} \ldots$



Safety/Reachability Properties

Reachability and safety properties are defined by a set of states $R \subseteq Q$.

We say that R is reachable in M if there exists a path $q_0 a_0 q_1 a_1 \dots q_n$ of M with $q_0 \in Q_0, q_n \in R$.

M is safe (with respect to R) if R is not reachable.

Alternatively, reachability and safety properties can be defined by sets of state labels $L \subseteq 2^{AP}$.

Then L is reachable in M if there exists a path $q_0 a_0 q_1 a_1 \dots q_n$ of M with $q_0 \in Q_0, \lambda(q_n) \in L$, and M is safe (with respect to L) if L is not reachable.

Example: Reachability Properties

Consider $R = \{q_5\}$.

R is reachable: $q_0\ start_r\ q_4\ start_r\ q_5$

M is not safe (wrt. R).

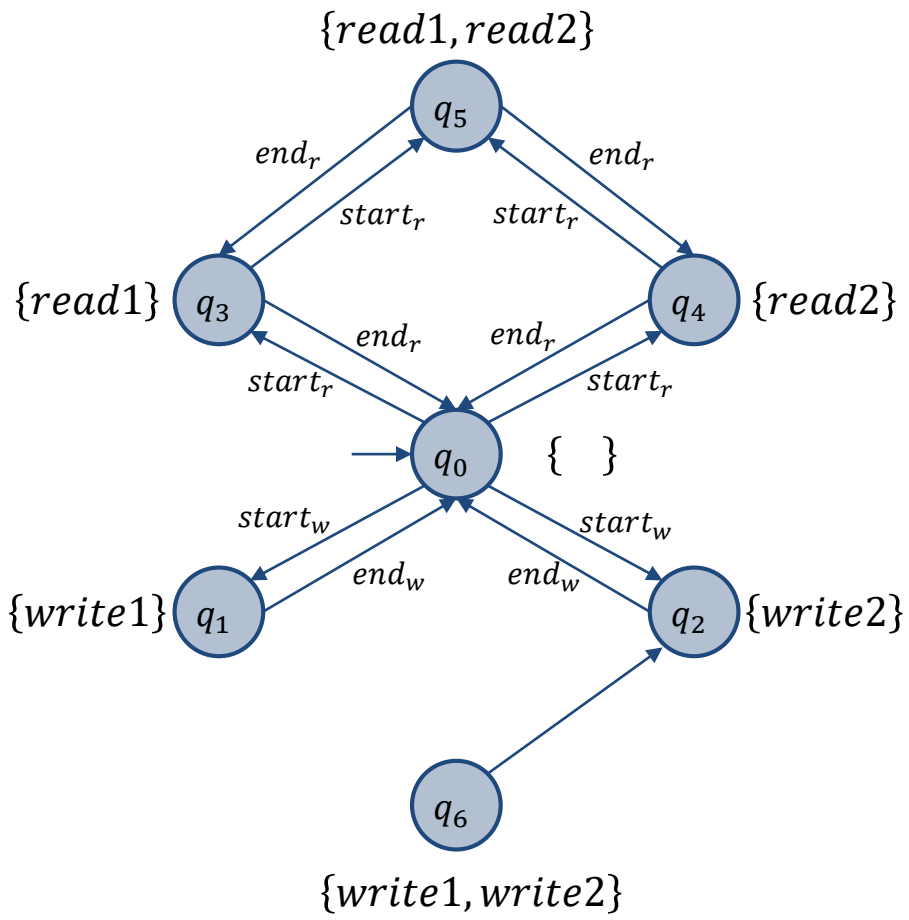
Consider $L = \{\ \{write1, write2\}\ \}$.

L is not reachable. There is not even a state with a label from L.

M is safe (wrt. L).

Consider a modified system M' with additional state q_6
that has no incoming transitions and $\lambda(q_6) = \{write1, write2\}$.

M' is safe wrt. L, and wrt. $R' = \{q_6\}$.



Decision Problems, Checking Safety/Reachability

A decision problem is a yes-or-no question, defined over an infinite set of inputs I.

Examples: Is $n \in \mathbb{N}$ a prime number? Does a given program terminate on input $i \in I$?

A decision problem is decidable if there exists an algorithm that can compute, for any input $i \in I$, whether the correct answer is yes or no.

Reachability of $R \subseteq Q$ in an LTS defines a decision problem:

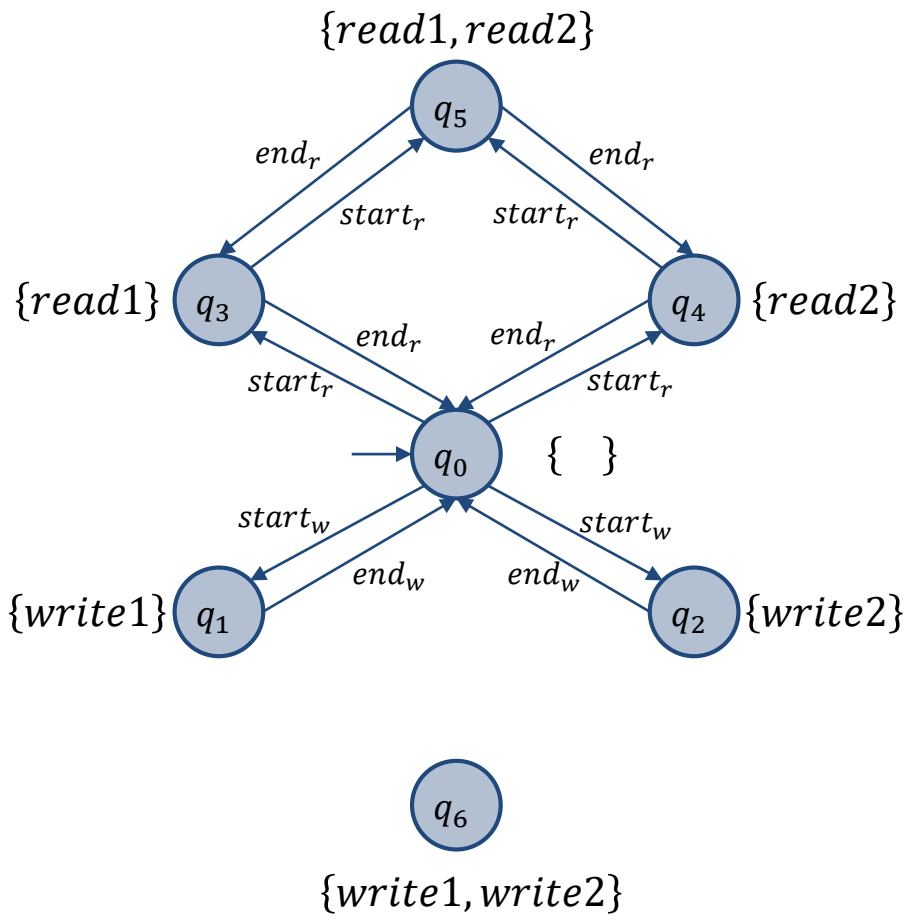
- the set of inputs is the set of all LTSs and subsets $R \subseteq Q$
- the answer is yes if and only if R is reachable in the given LTS.

Reachability of R in an LTS is decidable. It can be checked by explicit search (e.g., DFS or BFS) over the states of the LTS.

Example: Checking Reachability Properties

To check reachability of R in M, we can for example use the following algorithm, where $\delta(Reached)$ is the set of all states that are in $Reached$, or are reachable in one step of δ :

1. Let $Reached = Q_0$
2. Let $Reached' = \delta(Reached)$
3. If $R \cap Reached' \neq \emptyset$ then return "reachable"
4. If $Reached' \neq Reached$ then {let $Reached = Reached'$; goto 2}
5. Return "unreachable"



Composition by Synchronization: Two Processes

Let Σ_{int} be a set of internal actions and Σ_{sync} a set of synchronizing actions.

Then a process is an LTS $P = (Q, Q_0, \Sigma, \delta, \lambda)$ with $\Sigma = \Sigma_{int} \cup \{out_a, in_a \mid a \in \Sigma_{sync}\}$.

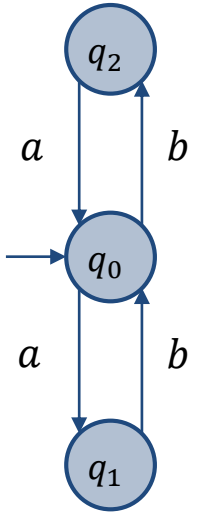
An action out_a is called an initiate action (or send action) and in_a is called a receive action.

The composition $P_1 \parallel P_2$ of two (uniform) processes is the LTS $\left(S, S_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta, \Lambda\right)$ with

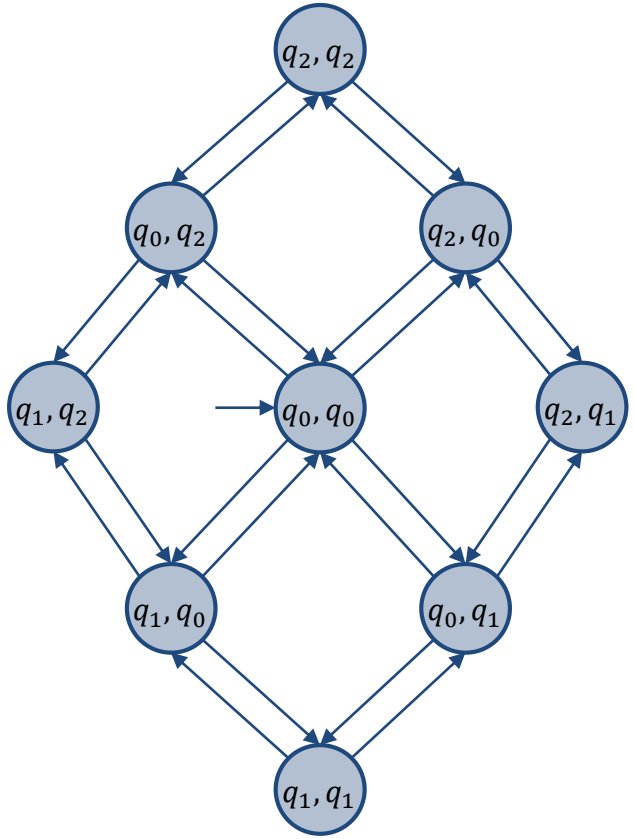
- $S = Q \times Q$ and $S_0 = Q_0 \times Q_0$
- $\Delta \subseteq S \times \left(\Sigma_{int} \cup \Sigma_{sync}\right) \times S$ is the set of all transitions that satisfy one of the following:
  - $\left((q_1, q_2), a, (q_1', q_2')\right) \in S \times \Sigma_{int} \times S$, where either $(q_1, a, q_1') \in \delta$ and $q_2 = q_2'$ or $(q_2, a, q_2') \in \delta$ and $q_1' = q_1$. This is called an internal transition.
  - $\left((q_1, q_2), a, (q_1', q_2')\right) \in S \times \Sigma_{sync} \times S$, where $(q_1, a_1, q_1') \in \delta$, $(q_2, a_2, q_2') \in \delta$, and either $a_1 = out_a$ and $a_2 = in_a$ or $a_1 = in_a$ and $a_2 = out_a$. This is called a synchronous transition.
- $\Lambda(q_1, q_2) = \{p_i \mid p \in AP \text{ and } p \in \lambda(q_i), 1 \leq i \leq 2\}$

Example: Composition of Two Processes

Process M



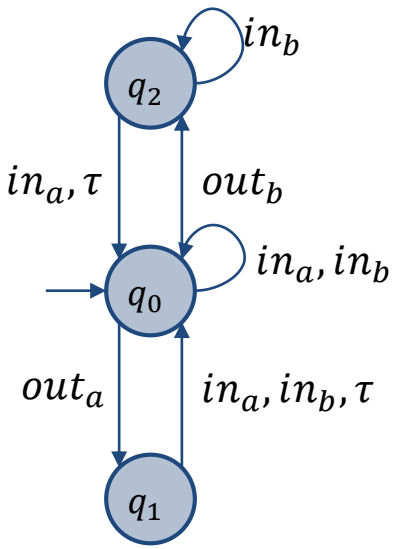
Composition $M_1 \parallel M_2$
for $\Sigma_{int} = \{a, b\}$, $\Sigma_{sync} = \{\quad\}$



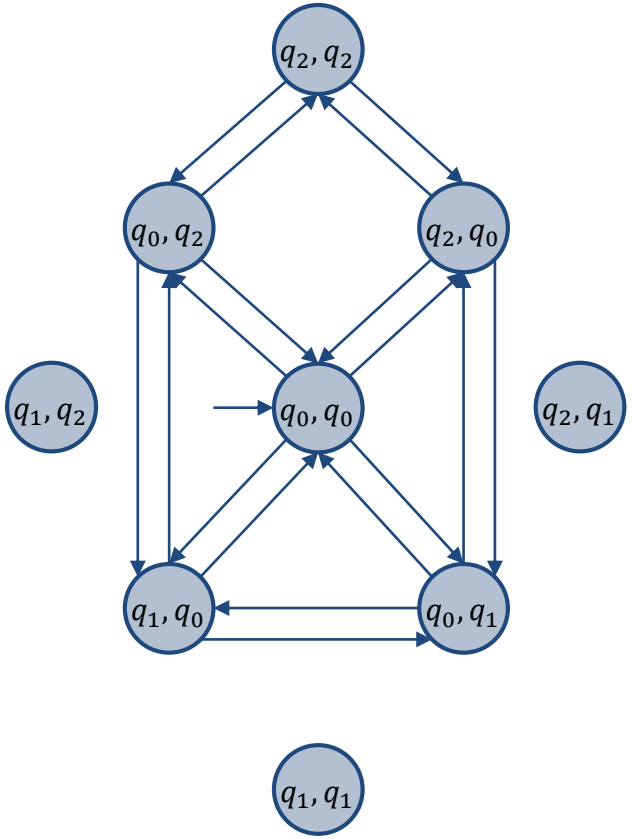
(omitting actions)

Example: Composition of Two Processes (II)

Process M



Composition $M_1 \parallel M_2$
for $\Sigma_{int} = \{\tau\}$, $\Sigma_{sync} = \{a, b\}$

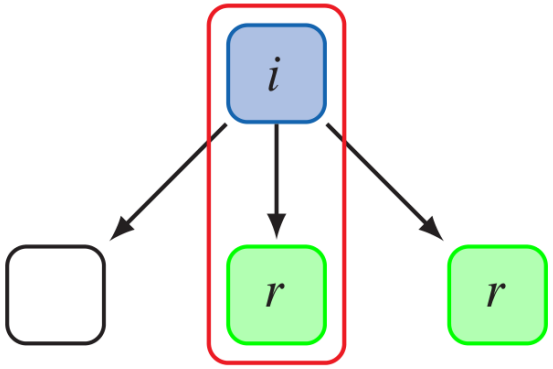


(omitting actions and self-loops)

Synchronization Primitives

For composition of two processes, we only need to consider pairwise synchronization.

In systems with more than two processes, this synchronization can schematically be given as

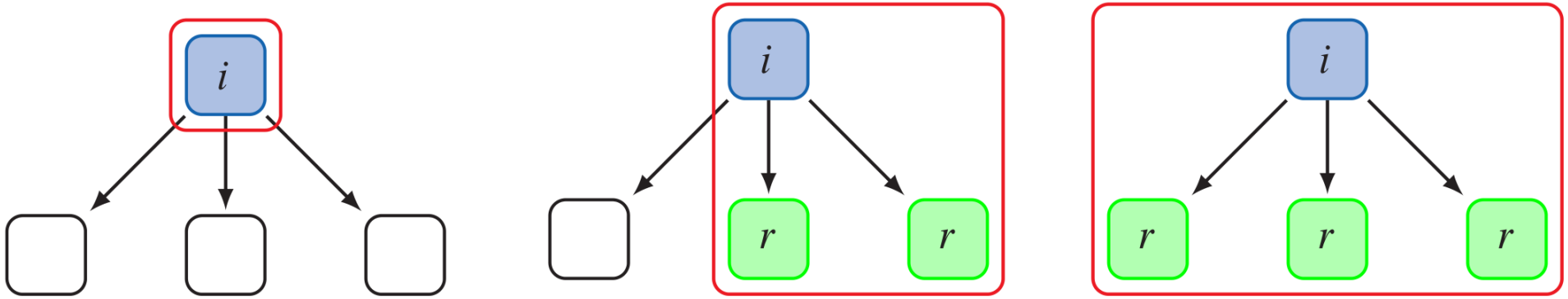


where i stands for a process that is ready to take action out_a, r stands for processes that are ready to take action in_a, and the red box shows which processes take the synchronous transition.

We call this type of synchronization pairwise rendezvous.

Synchronization Primitives (II)

In systems with more than two processes, other types of synchronization are possible. A prominent example is broadcast synchronization, which can schematically be given as



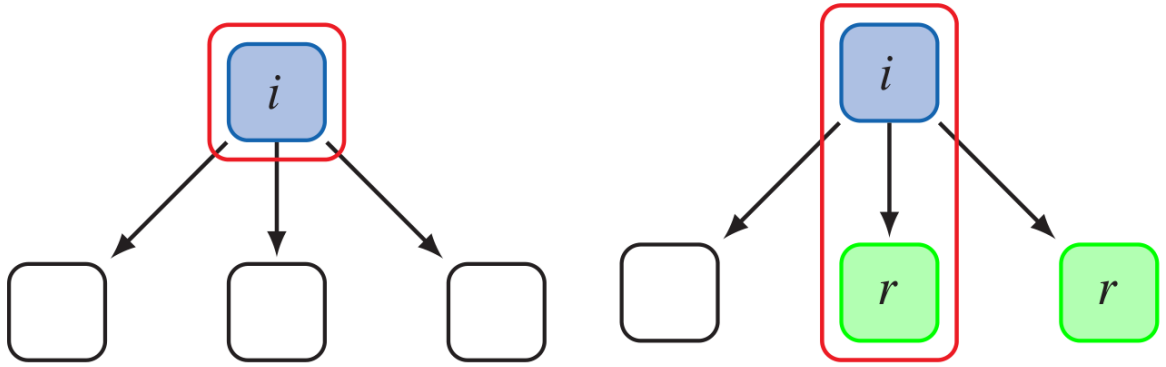
where i stands for a process that is ready to take action out_a, r stands for processes that are ready to take action in_a, and the red box shows which processes take the synchronous transition.

That is, exactly one process takes the initiating action, and all processes that are ready to take the receive action will do so.

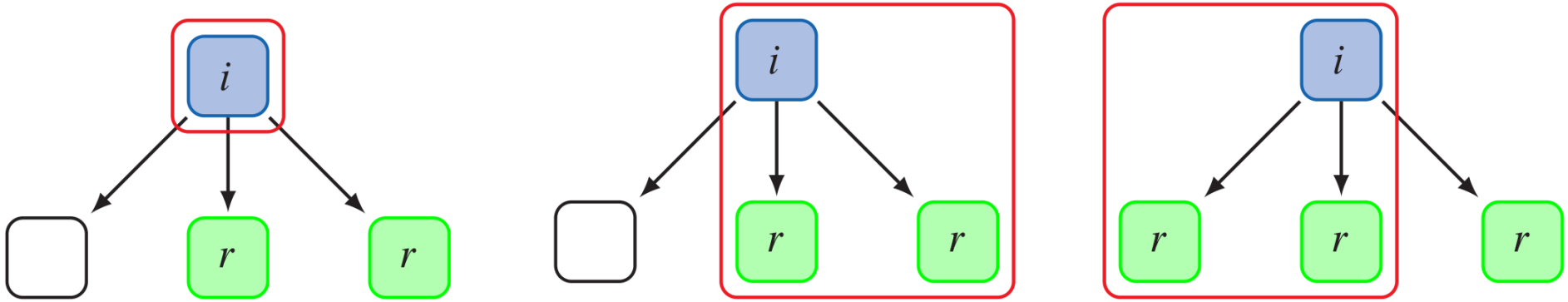
Synchronization Primitives (III)

Sometimes we also consider variants of pairwise rendezvous and broadcast.

In asynchronous rendezvous, an initiating transition can also be taken if there is no receiver:



In lossy broadcast, a potential receiver may or may not take the receive transition:



Composition by Synchronization: Many Processes

Let $P = (Q, Q_0, \Sigma, \delta, \lambda)$ be a process as before. For $s = (q_1, q_2, \dots, q_n) \in Q^n$, write $s(i)$ for q_i.
s is called a global state, the q_i are local states.

A synchronization constraint is a set $card \subseteq \mathbb{N}_0$.

The composition P^n of n (uniform) processes wrt. $card$ is the LTS $\left(S, S_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta, \Lambda\right)$ with

- $S = Q^n$
- $S_0 = (Q_0)^n$
- Δ as defined on the next slide
- $\Lambda(s) = \{p_i \mid p \in AP \text{ and } p \in \lambda\big(s(i)\big), i \in \{1, \dots, n\}\}$

Composition by Synchronization: Many Processes (II)

For the composition P^n of n (uniform) processes wrt. $card$,
$\Delta \subseteq S \times (\Sigma_{int} \cup \Sigma_{sync}) \times S$ is the set of all transitions that satisfy one of the following:

- an internal transition is an element $(s, a, s') \in S \times \Sigma_{int} \times S$ such that for some $i \in \{1, \dots, n\}$ $(s(i), a, s'(i)) \in \delta$, and $\mathrm{s(j)} = \mathrm{s'(j)}$ for $i \neq j \in \{1, \dots, n\}$
- a synchronous transition is an element $(s, a, s') \in S \times \Sigma_{sync} \times S$ such that for some $i \in \{1, \dots, n\}$ and some $I \subseteq \{1, \dots, n\} \setminus \{i\}$ with $|I| \in card$, we have
  - $s(i) \xrightarrow{out_a} s'(i)$ is (a local transition) in P
  - for every $j \in I$, $s(j) \xrightarrow{in_a} s'(j)$ is (a local transition) in P
  - for every $j \in \{1, \dots, n\} \setminus (I \cup \{i\})$, $s'(j) = s(j)$
  - I is maximal, i.e., there does not exist $I' \supset I$ with $|I'| \in card$ and such that for all $j \in I'$ there is a local transition from $s(j)$ with action in_a.

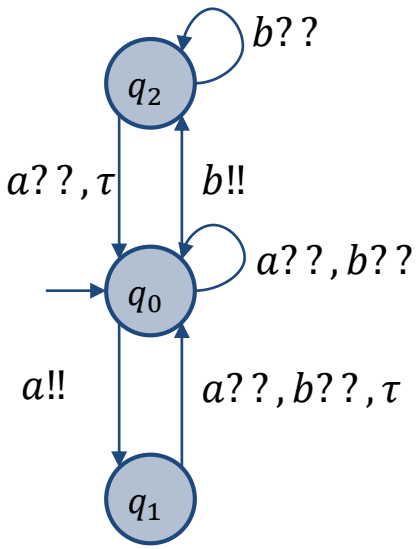
Example: Synchronization by Broadcast

$card = \mathbb{N}_0$

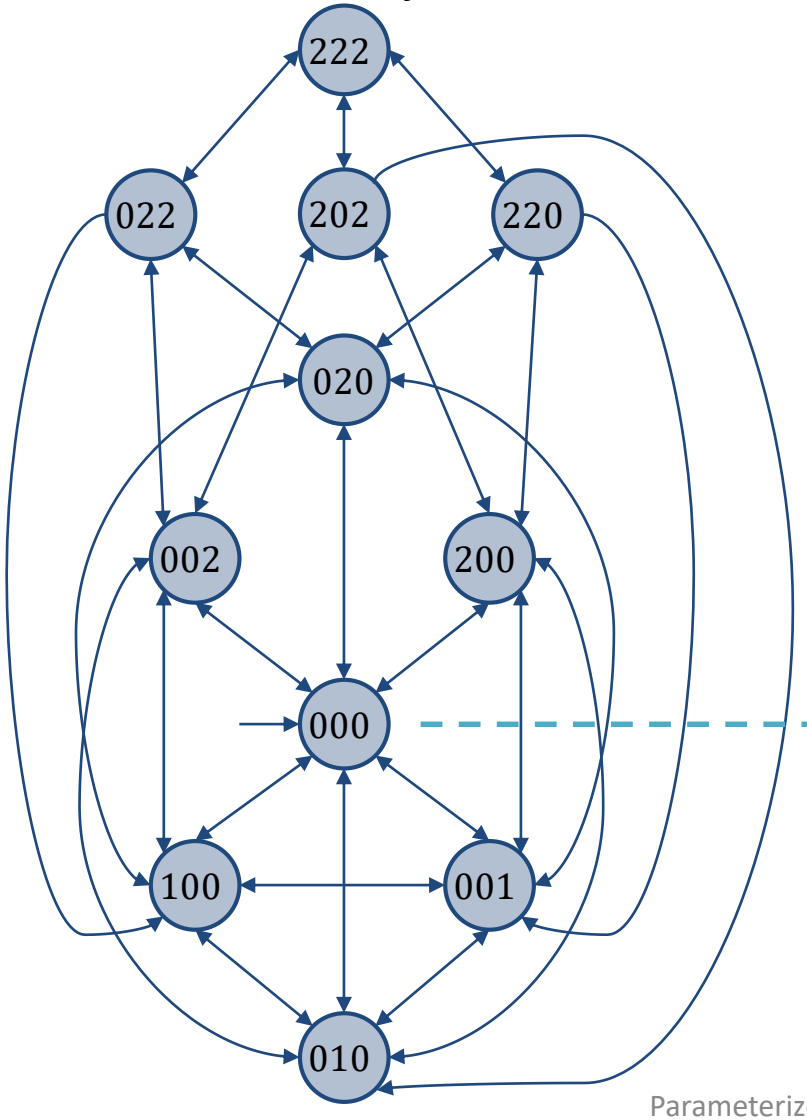
$a!! = out_a$

$a?? = in_a$

Process M



Composition M^3
for $\Sigma_{int} = \{\tau\}$, $\Sigma_{sync} = \{a, b\}$



"000" stands for global state (q_0, q_0, q_0)

Composition by Synchronization: Discussion

Notation: for the composition P^n wrt. $card$ we also write P^n_{card} if $card$ is not clear from the context

This definition of composition is very general.

$card$ allows us to define many types of synchronization, e.g.,

- pairwise rendezvous: $card = \{1\}$
- broadcast: $card = \mathbb{N}_0$
- asynchronous rendezvous: $card = \{0,1\}$

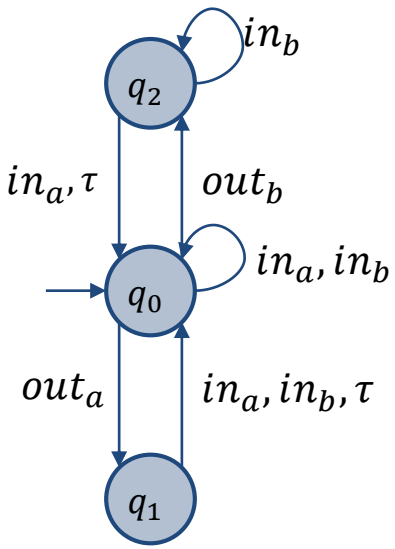
"Composition of two processes"
is the same as
"composition of many processes"
with $n = 2, card = \{1\}$

Modification of the definition allows us to capture even more synchronization primitives, e.g.,

- lossy broadcast: $card = \mathbb{N}_0$ and drop condition that I must be maximal in synchr. transitions

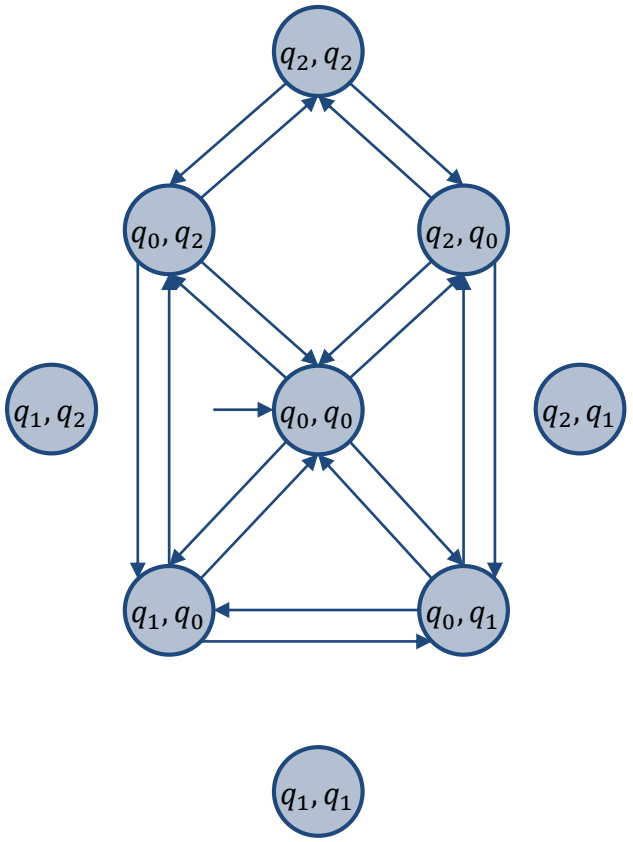
Example: Composition of Two Processes (III)

Process M



how does the system change if we remove internal transition $q_2 \xrightarrow{\tau} q_0$?

Composition $M_1 \parallel M_2$
for $\Sigma_{int} = \{\tau\}, \Sigma_{sync} = \{a, b\}, \text{card} = \{1\}$



a run that enters (q_2, q_2) will be deadlocked!

(omitting actions and self-loops)

Deadlock Detection

Composition of processes can introduce deadlocks, depending on the synchronization primitive (expressed by $card$).

Reachability of a deadlock state is a special specification that we may use in verification:

can we reach a global state where no transition is possible?

Closing remarks

- Formal models are essential for what we will do throughout the course
- Models and formalisms seen today are complex because they are general
- Starting next week, we look at models for specific types of systems, which can be simpler